```
builder.Services.AddDefaultIdentity<CityBreaksUser>()
    .AddEntityFrameworkStores<CityBreaksContext>();
```

This service registration configures Identity with its default settings. You will see what those settings are and how to customize them a bit later.

9.2.3 Adding migration

Finally, you are at the stage where you can add a migration that will scaffold the tables required in the database. Like in the last chapter, you create a migration either through the Package Manager Console using the `add-migration` command or through `dotnet ef migrations add` via the CLI. You will name this migration `AddedIdentity`:

```
[PMC]
add-migration AddedIdentity
[CLI]
dotnet ef migrations add AddedIdentity
```

Once this command has been executed, take a moment to examine the code in the migration's `Up` method. If all has gone according to plan, you should have code that will create seven tables, each one representing a `DbSet` in the `IdentityDbContext` class. Apply the migration using the `update-database` command or `dotnet ef database update` via the CLI, and then take a look at the revised schema of your database. Sure enough, seven tables have been added to the database (figure 9.4), each one's name prefixed with `AspNet`.